

Architecture of DDiT and ContDIT

From High-Level Overview to Full Mathematical Detail

Auto-generated from CANDI Diffusion Codebase

June 30, 2026

Contents

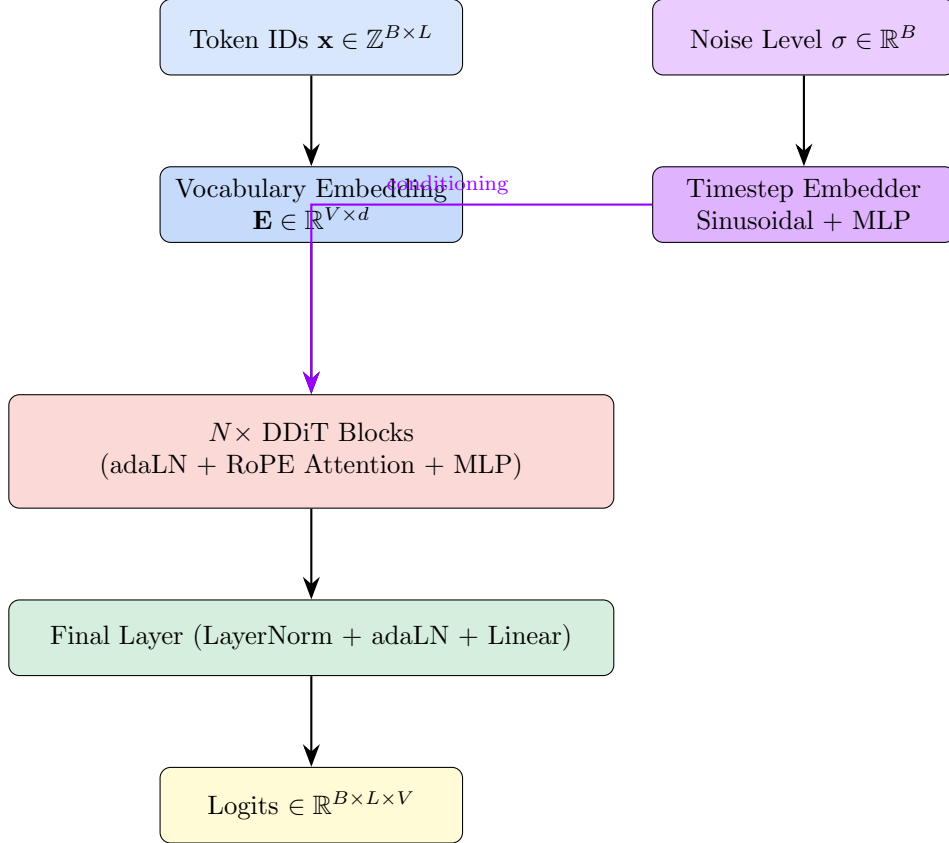
1	Level 1: High-Level Overview	2
1.1	DDiT — Discrete Diffusion Transformer	2
1.2	ContDIT — Continuous + Discrete Diffusion Transformer	2
2	Level 2: DDiT Block Internals	4
2.1	DDiT Block with adaLN	5
3	Level 2: ContDIT Hybrid Input Construction	6
3.1	Timestep Conditioning in ContDIT	6
4	Level 3: Full Mathematical Detail	7
4.1	Embedding Layer	7
4.1.1	DDiT (dit.py)	7
4.1.2	ContDIT (dit_cont.py)	7
4.2	Timestep Embedder	7
4.3	Rotary Position Embeddings (RoPE)	7
4.4	Layer Normalization	7
4.5	Adaptive Layer Normalization (adaLN)	8
4.6	DDiT Block — Full Math	8
4.7	Final Output Layer (DDiTFinalLayer)	8
5	Level 3: ContDIT Full Forward Pass	9
5.1	Complete Forward Pass Algorithm	9
6	Level 3: Detailed Component Diagrams	10
6.1	Attention Mechanism Detail	10
6.2	MLP Detail	10
6.3	Bias-Dropout-Scale Residual Connection	10
7	Summary: DDiT vs ContDIT Comparison	11
8	Complete Data Flow: ContDIT End-to-End	12
9	Initialization and Implementation Notes	13
9.1	Zero Initialization Strategy	13
9.2	Precision and Numerical Stability	13
9.3	Hyperparameters	13

1 Level 1: High-Level Overview

Both DDiT and ContDIT are **Diffusion Transformers** for discrete text generation. They share the same internal transformer block structure but differ in how inputs are prepared and what noise processes they handle.

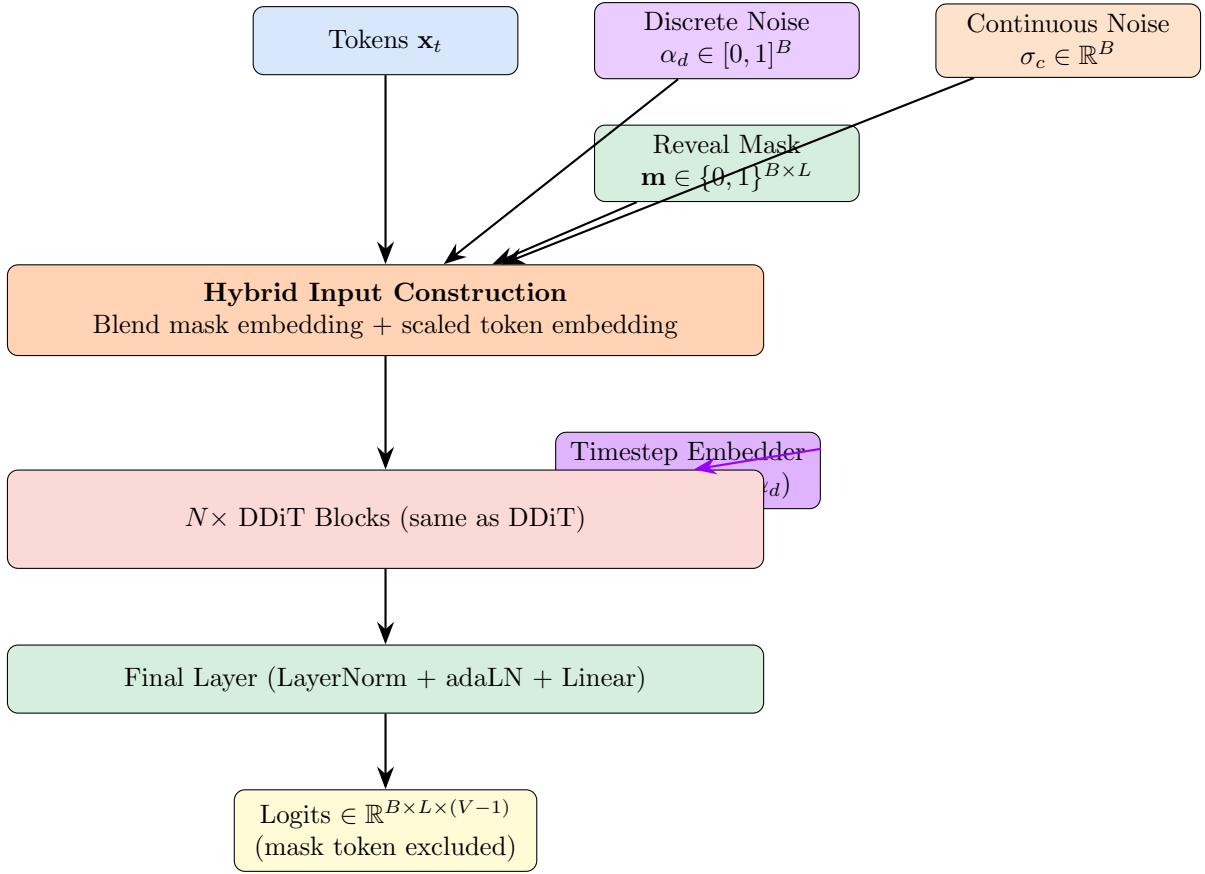
1.1 DDiT — Discrete Diffusion Transformer

DDiT operates on sequences corrupted by **discrete masking** only. It takes token IDs and a scalar noise level σ , and predicts logits over the vocabulary.



1.2 ContDIT — Continuous + Discrete Diffusion Transformer

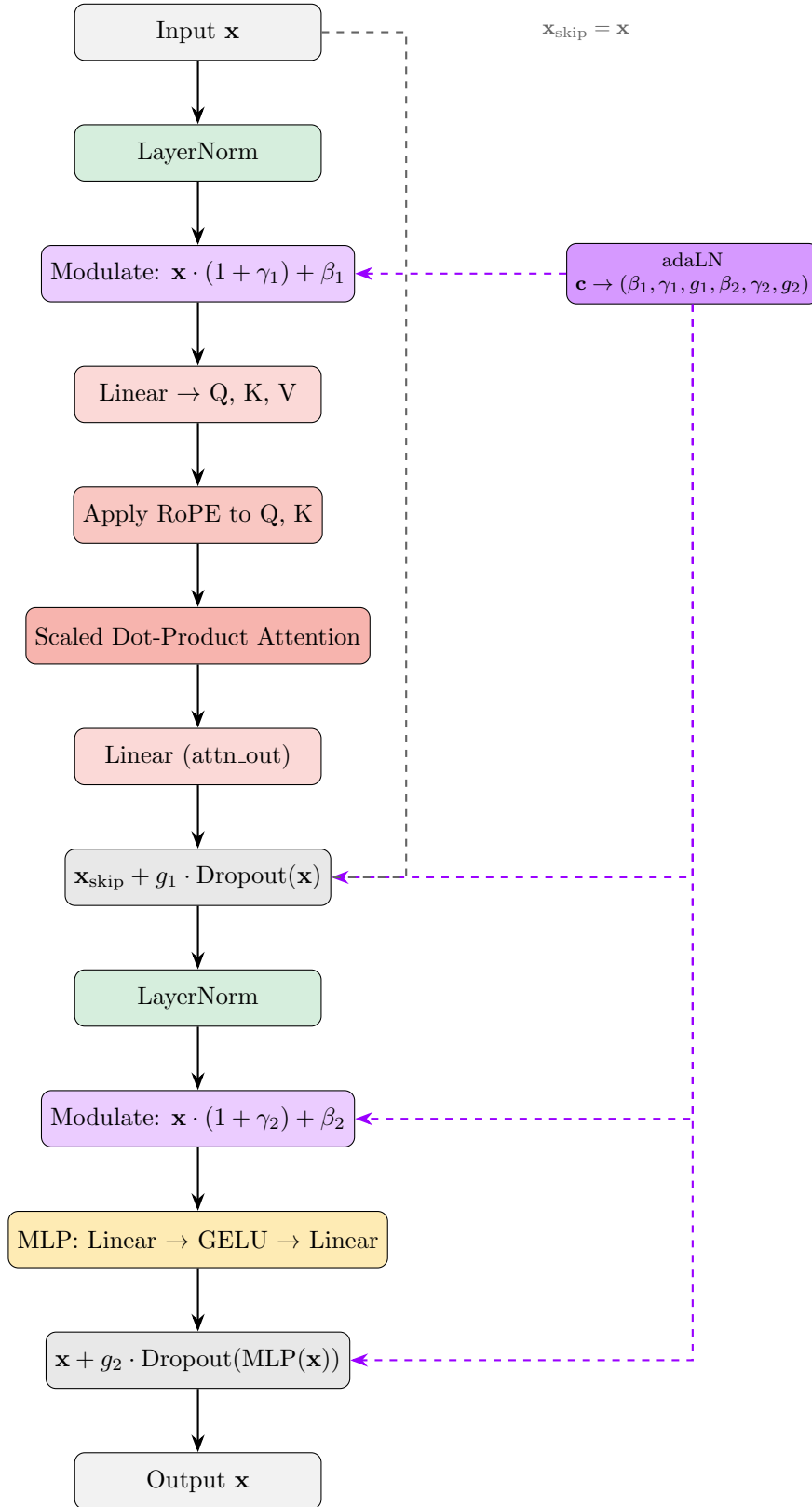
ContDIT handles CANDI's **hybrid noise process**: tokens are corrupted by both discrete masking **and** continuous Gaussian noise in embedding space.



2 Level 2: DDiT Block Internals

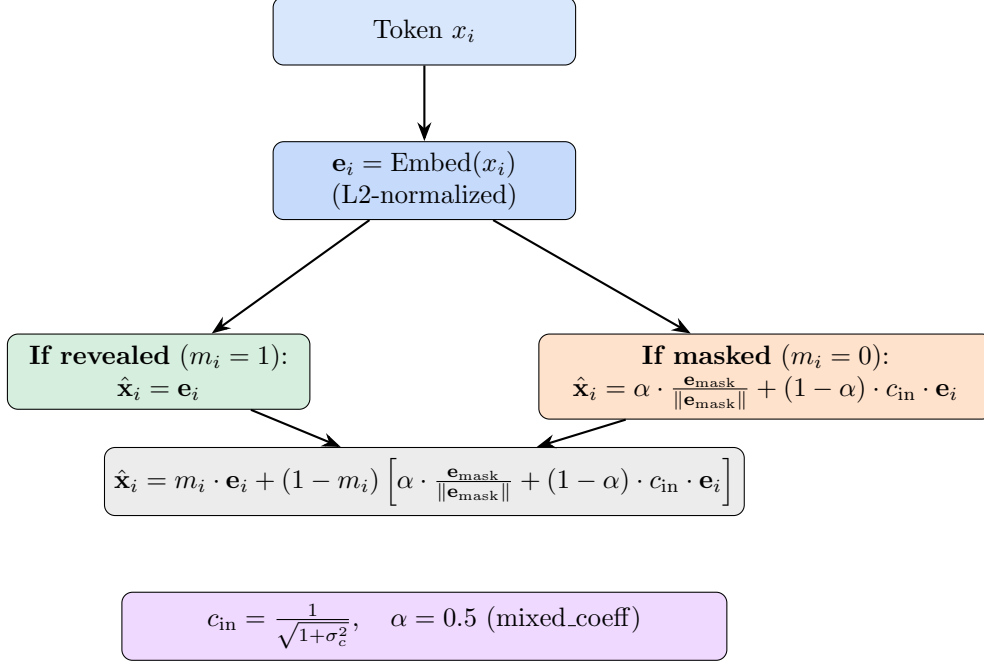
Each DDiT Block consists of two sub-layers: **Multi-Head Self-Attention** (with RoPE) and a **Feed-Forward Network** (MLP), both conditioned via **Adaptive Layer Normalization** (adaLN).

2.1 DDiT Block with adaLN



3 Level 2: ContDIT Hybrid Input Construction

The key difference in ContDIT is its input preparation. For each position, the model constructs a hybrid representation depending on whether that position is **revealed** (already denoised) or **masked** (still corrupted).



3.1 Timestep Conditioning in ContDIT

The discrete noise probability α_d is converted to a log-scale sigma before being passed to the same `TimestepEmbedder`:

$$\sigma_{\text{discrete}} = -\log(1 - \alpha_d)$$

$$\mathbf{c} = \text{SiLU}(\text{TimestepEmbedder}(\sigma_{\text{discrete}})) \in \mathbb{R}^{d_{\text{cond}}}$$

4 Level 3: Full Mathematical Detail

4.1 Embedding Layer

4.1.1 DDiT (dit.py)

Standard learned embedding with Kaiming initialization:

$$\mathbf{E} \in \mathbb{R}^{V \times d}, \quad \mathbf{e}_i = \mathbf{E}[x_i]$$

For soft inputs (probability distributions over vocab):

$$\mathbf{e} = \text{softmax}(\mathbf{p}) \cdot \mathbf{E}$$

4.1.2 ContDIT (dit_cont.py)

L2-normalized embedding (unit norm per token):

$$\hat{\mathbf{E}}[i] = \frac{\mathbf{E}[i]}{\|\mathbf{E}[i]\|_2}, \quad i \in \{0, \dots, V-2\}$$

Note: only the first $V-1$ rows are normalized (mask token excluded from normalization in this variant).

4.2 Timestep Embedder

Sinusoidal positional encoding followed by a 2-layer MLP:

$$\begin{aligned} \text{freq}_k &= \exp\left(-\frac{\log(10000) \cdot k}{D/2}\right), \quad k = 0, \dots, \frac{D}{2} - 1 \\ \mathbf{t}_{\text{freq}} &= \left[\cos(t \cdot \text{freq}_0), \dots, \cos(t \cdot \text{freq}_{D/2-1}), \sin(t \cdot \text{freq}_0), \dots, \sin(t \cdot \text{freq}_{D/2-1}) \right] \\ \mathbf{t}_{\text{emb}} &= W_2 \cdot \text{SiLU}(W_1 \cdot \mathbf{t}_{\text{freq}} + b_1) + b_2 \in \mathbb{R}^{d_{\text{cond}}} \end{aligned}$$

In the main forward pass, an additional SiLU activation is applied:

$$\mathbf{c} = \text{SiLU}(\mathbf{t}_{\text{emb}})$$

4.3 Rotary Position Embeddings (RoPE)

For position p and dimension pair $(2k, 2k+1)$:

$$\begin{aligned} \text{RoPE}(\mathbf{q}, p)_{2k} &= q_{2k} \cos(p \cdot \theta_k) - q_{2k+1} \sin(p \cdot \theta_k) \\ \text{RoPE}(\mathbf{q}, p)_{2k+1} &= q_{2k} \sin(p \cdot \theta_k) + q_{2k+1} \cos(p \cdot \theta_k) \end{aligned}$$

where $\theta_k = 10000^{-2k/d_{\text{head}}}$.

Applied to Q and K only (V is unchanged).

4.4 Layer Normalization

Custom implementation (cast to float32 for numerical stability):

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where γ is a learnable weight (no bias), and the normalization is computed in float32.

4.5 Adaptive Layer Normalization (adaLN)

The conditioning vector $\mathbf{c}$ is projected to 6 modulation parameters per block:

$$(\beta_1, \gamma_1, g_1, \beta_2, \gamma_2, g_2) = \text{Linear}(\mathbf{c}) \in \mathbb{R}^{6d}$$

Initialized to zero (identity at start of training).

The modulation function:

$$\text{Modulate}(\mathbf{x}, \beta, \gamma) = \mathbf{x} \odot (1 + \gamma) + \beta$$

4.6 DDiT Block — Full Math

Given input $\mathbf{x} \in \mathbb{R}^{B \times L \times d}$ and conditioning $\mathbf{c} \in \mathbb{R}^{B \times d_{\text{cond}}}$:

1. **Save residual:** $\mathbf{x}_{\text{skip}} = \mathbf{x}$

2. **Pre-norm + modulate (attention):**

$$\mathbf{x}' = \text{Modulate}(\text{LayerNorm}(\mathbf{x}), \beta_1, \gamma_1)$$

3. **QKV projection:**

$$[\mathbf{Q}, \mathbf{K}, \mathbf{V}] = \mathbf{x}' W_{\text{qkv}} \in \mathbb{R}^{B \times L \times 3d}$$

Reshaped to $B \times L \times H \times d_h$ per head.

4. **RoPE:** Apply rotary embeddings to Q and K.

5. **Scaled dot-product attention** (bidirectional, no mask):

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right) \mathbf{V}$$

6. **Output projection + gated residual:**

$$\mathbf{x} = \mathbf{x}_{\text{skip}} + g_1 \odot \text{Dropout}(W_{\text{out}} \cdot \text{Attn}(\dots))$$

7. **Pre-norm + modulate (MLP):**

$$\mathbf{x}' = \text{Modulate}(\text{LayerNorm}(\mathbf{x}), \beta_2, \gamma_2)$$

8. **MLP:**

$$\text{MLP}(\mathbf{x}') = W_2 \cdot \text{GELU}_{\tanh}(W_1 \mathbf{x}' + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{4d \times d}$, $W_2 \in \mathbb{R}^{d \times 4d}$.

9. **Gated residual:**

$$\mathbf{x} = \mathbf{x} + g_2 \odot \text{Dropout}(\text{MLP}(\mathbf{x}'))$$

4.7 Final Output Layer (DDiTFinalLayer)

1. **LayerNorm:** $\mathbf{x}' = \text{LayerNorm}(\mathbf{x})$

2. **adaLN modulation** (2 parameters):

$$(\beta_f, \gamma_f) = \text{Linear}(\mathbf{c}) \in \mathbb{R}^{2d}$$

$$\mathbf{x}'' = \mathbf{x}' \odot (1 + \gamma_f) + \beta_f$$

3. **Linear projection to logits:**

$$\text{logits} = W_{\text{out}} \mathbf{x}'' + b_{\text{out}} \in \mathbb{R}^{B \times L \times V}$$

Initialized to zero (model starts as identity).

5 Level 3: ContDIT Full Forward Pass

5.1 Complete Forward Pass Algorithm

Inputs:

- $\mathbf{x}_t \in \mathbb{Z}^{B \times L}$ — corrupted token IDs
- $\alpha_d \in [0, 1]^B$ — discrete noise level (masking probability)
- $\sigma_c \in \mathbb{R}_+^B$ — continuous noise standard deviation
- $\mathbf{m} \in \{0, 1\}^{B \times L}$ — reveal mask (1 = clean, 0 = still masked)
- (optional) $\tilde{\mathbf{E}} \in \mathbb{R}^{B \times L \times d}$ — pre-computed noisy embeddings

Step 1: Compute scaling factors

$$\sigma_{\text{disc}} = -\log(1 - \alpha_d) \in \mathbb{R}^B$$

$$c_{\text{in}} = \frac{1}{\sqrt{1 + \sigma_c^2}} \in \mathbb{R}^B$$

Step 2: Token embedding (L2-normalized)

$$\mathbf{e}_i = \frac{\mathbf{E}[x_{t,i}]}{\|\mathbf{E}[x_{t,i}]\|_2} \in \mathbb{R}^d$$

Step 3: Mask embedding (normalized, from last row of $\mathbf{E}$)

$$\mathbf{e}_{\text{mask}} = \frac{\mathbf{E}[V]}{\|\mathbf{E}[V]\|_2} \in \mathbb{R}^d$$

(broadcast to $B \times L \times d$)

Step 4: Hybrid input construction

$$\hat{\mathbf{x}}_i = m_i \cdot \mathbf{e}_i + (1 - m_i) \cdot [\alpha \cdot \mathbf{e}_{\text{mask}} + (1 - \alpha) \cdot c_{\text{in}} \cdot \mathbf{e}_i]$$

where $\alpha = 0.5$ is the `mixed_coeff` hyperparameter.

If external noisy embeddings $\tilde{\mathbf{E}}$ are provided:

$$\hat{\mathbf{x}}_i = m_i \cdot \mathbf{e}_i + (1 - m_i) \cdot [\alpha \cdot \mathbf{e}_{\text{mask}} + (1 - \alpha) \cdot c_{\text{in}} \cdot \tilde{\mathbf{E}}_i]$$

Step 5: Timestep conditioning

$$\mathbf{c} = \text{SiLU}(\text{TimestepEmbedder}(\sigma_{\text{disc}})) \in \mathbb{R}^{B \times d_{\text{cond}}}$$

Step 6: Rotary embeddings

$$(\cos \Theta, \sin \Theta) = \text{Rotary}(\hat{\mathbf{x}})$$

Step 7: Transformer blocks (in bfloat16)

$$\hat{\mathbf{x}} \leftarrow \text{DDiTBlock}_i(\hat{\mathbf{x}}, (\cos \Theta, \sin \Theta), \mathbf{c}), \quad i = 1, \dots, N$$

Step 8: Final layer

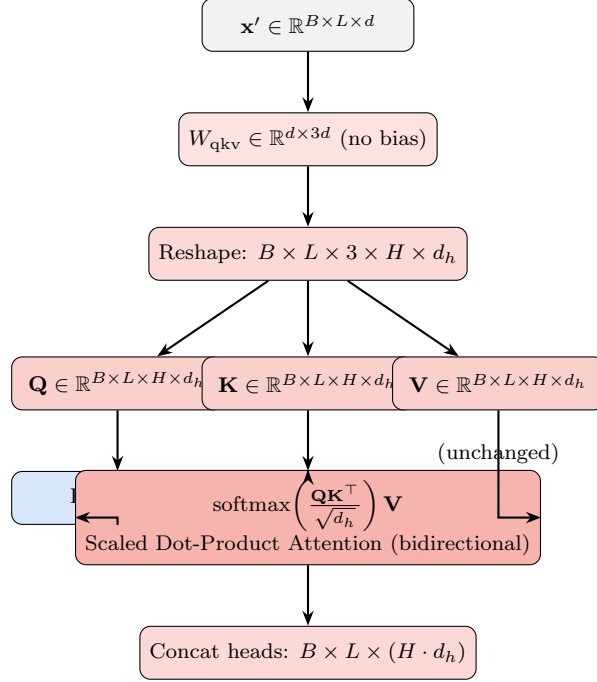
$$\text{logits} = \text{DDiTFinalLayer}(\hat{\mathbf{x}}, \mathbf{c}) \in \mathbb{R}^{B \times L \times V}$$

Step 9: Exclude mask token

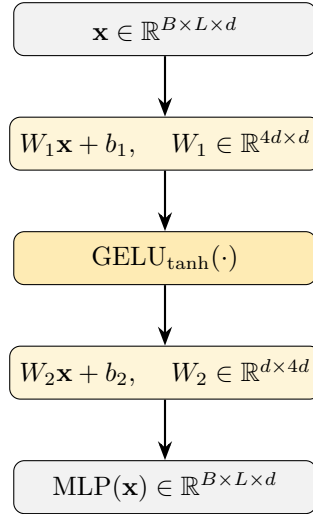
$$\text{output} = \text{logits}[:, :, : V-1] \in \mathbb{R}^{B \times L \times (V-1)}$$

6 Level 3: Detailed Component Diagrams

6.1 Attention Mechanism Detail



6.2 MLP Detail



where:

$$\text{GELU}_{\tanh}(x) = 0.5 x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715 x^3) \right] \right)$$

6.3 Bias-Dropout-Scale Residual Connection

The fused kernel computes:

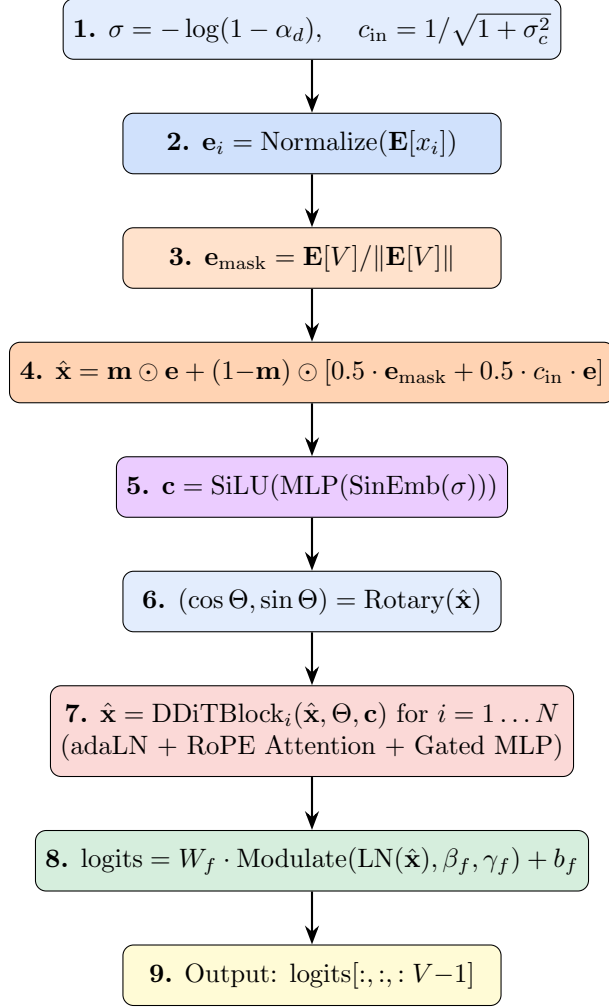
$$\text{BiasDropoutScale}(\mathbf{x}, \text{bias}, \text{scale}, \mathbf{x}_{\text{skip}}, p) = \mathbf{x}_{\text{skip}} + \text{scale} \odot \text{Dropout}(\mathbf{x} + \text{bias}, p)$$

In adaLN mode, the “scale” is the **gate** parameter $g \in \mathbb{R}^d$ from the conditioning.

7 Summary: DDiT vs ContDIT Comparison

Component	DDiT	ContDIT
Algorithm	MDLM / SEDD (pure discrete)	CANDI (hybrid discrete + continuous)
Input tokens	$\mathbf{x} \in \mathbb{Z}^{B \times L}$	$\mathbf{x}_t \in \mathbb{Z}^{B \times L}$
Noise inputs	$\sigma \in \mathbb{R}^B$ (scalar)	α_d, σ_c , reveal mask $\mathbf{m}$
Embedding	$\mathbf{E}[x]$ (standard)	$\mathbf{E}[x] / \ \mathbf{E}[x]\ _2$ (L2-normalized)
Input construction	Direct embedding lookup	Hybrid blend of mask emb + scaled token emb
Timestep σ	Passed directly	$\sigma = -\log(1 - \alpha_d)$
Continuous scaling	N/A	$c_{\text{in}} = 1 / \sqrt{1 + \sigma_c^2}$
Transformer blocks	DDiTBlock (adaLN + RoPE + MLP)	Same DDiTBlock
Output dim	V (full vocab)	$V - 1$ (mask token excluded)
Initialization	adaLN & final layer $\rightarrow$ zero	Same

8 Complete Data Flow: ContDIT End-to-End



9 Initialization and Implementation Notes

9.1 Zero Initialization Strategy

Key layers are initialized to **zero** so the model starts as an identity function:

- `adaLN_modulation` (weight and bias) $\rightarrow 0$ in every `DDiTBlock`
- `DDiTFinalLayer.linear` (weight and bias) $\rightarrow 0$
- `DDiTFinalLayer.adaLN_modulation` (weight and bias) $\rightarrow 0$

This means at initialization:

- All gates $g_1, g_2 = 0 \Rightarrow$ residual connections are pure skip
- All shifts $\beta = 0$, scales $\gamma = 0 \Rightarrow$ modulation is identity
- Final projection outputs all zeros

9.2 Precision and Numerical Stability

- **LayerNorm**: Cast to float32 internally for stability
- **RoPE**: Applied in float32 (autocast disabled)
- **Transformer blocks**: Run in bfloat16 via `torch.cuda.amp.autocast`
- **Embedding normalization** (ContDIT): Computed in float32

9.3 Hyperparameters

Parameter	Symbol	Typical Value
Hidden dimension	d	<code>config.hidden_size</code>
Conditioning dimension	d_{cond}	<code>config.cond_dim</code>
Number of heads	H	<code>config.n_heads</code>
Head dimension	$d_h = d/H$	—
Number of blocks	N	<code>config.n_blocks</code>
MLP ratio	—	4
Dropout	p	<code>config.dropout</code>
Mixed coefficient (ContDIT)	α	0.5
Frequency embedding dim	D	256
RoPE base	—	10,000